SRI SHAKTHI
INSTITUTE OF ENGINEERING AND TECHNOLOGY
An Autonomous Institution
Coimbatore - 62.
POWERING THE YOUTH
EMPOWERING THE NATION
NBA NATIONAL BOARD OF ACCREDITATION For CSE,MECH,EEE, IT
NAAC

MERN PROJECT

POCKET LAWYER
A LEGAL DOCUMENT ASSISTANT

DONE BY:
RADIKAA R
III - CSE - C

ABSTRACT

Pocket Lawyer is a MERN-based legal document assistant designed to help users understand and interact with their legal documents through document upload, document-based question answering, legal topic exploration, and personalized document management. Instead of manually reading lengthy legal documents, users can upload supported files and ask questions based on the content of the selected document.

The system provides a personalized workspace where users can register and log in, upload legal documents, view their uploaded documents, select a document, and ask questions related to its contents. The application focuses on providing answers based on the uploaded document rather than treating every question as a general web search.

The platform is implemented using the MERN stack, with React.js responsible for the interactive frontend, Node.js and Express.js providing the backend REST APIs, and MongoDB serving as the database. Mongoose is used as the object data modeling layer between the backend and MongoDB..

Pocket Lawyer supports document formats including PDF, DOC, DOCX, and TXT, with a maximum upload size of 10 MB. The uploaded document is processed and its text is extracted and stored for document-based question answering.

The system is organized into core modules including User Authentication and Management, Document Upload and Management, Document Text Extraction, Document-Based Question Answering, Legal Topics and Search, Bookmarks and Search History, and User Dashboard and Document Workspace. By combining these modules, Pocket Lawyer provides an integrated platform for users to work with legal documents in a simple and organized manner.

TECH STACK

Frontend

Pocket Lawyer's frontend is built using React.js, providing a component-based architecture for creating an interactive and responsive user interface.

React manages different views of the application, including authentication, dashboard, document upload, document listing, document details, legal topics, and document-based question answering.

The frontend is structured using reusable components so that features such as navigation bars, authentication forms, document cards, upload forms, question panels, answer sections, dashboards, and legal-topic sections can be developed and maintained independently.

The document interface is a central part of the frontend. It allows users to view their uploaded documents, select a document, and interact with the question-answering functionality. The frontend communicates with the backend through REST APIs. User authentication information is used to protect personal documents and user-specific operations.

Backend

The backend of Pocket Lawyer is developed using Node.js and Express.js, providing the REST API layer between the React frontend and MongoDB.

The backend follows a modular structure in which different application responsibilities are handled through separate routes, controllers, models, middleware, and services rather than placing all functionality into a single server file.

The backend manages user authentication, document upload, document retrieval, document processing, question answering, legal topics, bookmarks, and search history.
REST APIs allow the frontend to register and authenticate users, upload documents, retrieve documents, submit questions, retrieve answers, and manage user-specific information.
Node.js provides the runtime environment for the server-side application, while Express.js handles HTTP requests, routing, middleware, and API communication.

This separation between the frontend and backend allows the user interface, business logic, database operations, and document-processing functionality to be developed independently.

Database

Pocket Lawyer uses MongoDB as its primary database for storing application data.

MongoDB stores different types of information required by the application, including:

- User accounts
- Uploaded documents
- Extracted document text
- Document questions and answers
- Legal topics
- Bookmarks
- Search history

Mongoose is used as the object data modeling layer between Node.js and MongoDB. It provides schemas and models that define the structure of application data and simplifies database operations from the Express.js backend.

The database associates documents and questions with their respective users so that each user can maintain a personalized legal document workspace.

The stored document information allows users to access their previously uploaded documents and continue asking questions about them across different sessions.

Document Processing Engine

Document processing is one of the core technical components of Pocket Lawyer.

When a supported legal document is uploaded, the backend processes the file and extracts its readable text. The extracted text is associated with the uploaded document and stored for later retrieval.

Pocket Lawyer supports the following document formats:

- PDF
- DOC
- DOCX
- TXT

The maximum supported upload size is **10 MB**.

The extracted document text becomes the primary source for the document-based question-answering system.

This approach allows the application to process the content of a user's own legal document instead of requiring the user to manually search through a lengthy document.

Document-Based Question Answering

The Document-Based Question Answering module is one of the main features of Pocket Lawyer.

Users can select one of their uploaded documents and enter a question related to its contents.

The backend receives the selected document and question, retrieves the relevant content from the stored document text, and identifies passages that are related to the question.

The relevant passages are then used to produce an answer based on the selected document.

The system is designed so that the question-answering process remains focused on the selected document.

The result can contain:

- User question
- Relevant document information
- Generated answer
- Associated document
- Question history
-

The question and answer information can be stored in MongoDB so that users can review their previous interactions.

API Layer

The Pocket Lawyer REST API is organized around the major resources and functions of the application.

The backend provides endpoints for:

- User authentication
- Document upload
- Document retrieval
- Document-based questions
- Legal topics
- Bookmarks
- Search history

The API layer acts as the communication bridge between the React frontend and MongoDB.

When a user performs an operation through the interface, the frontend sends the appropriate request to the Express.js server.

The backend processes the request, verifies authentication and authorization, performs the required business logic, interacts with MongoDB through Mongoose, and returns the required response to the frontend.

This architecture keeps database operations and application logic on the server while allowing the React frontend to focus primarily on presentation and user interaction.

Authentication, document access, question processing, and database operations remain protected within the backend.

MERN Architecture

Pocket Lawyer follows the MERN stack architecture, consisting of four primary technologies:

MongoDB

Stores users, documents, extracted text, document questions, legal topics, bookmarks, and search history.

Express.js

Provides the REST API, server-side routing, middleware, and request handling.

React.js

Builds the interactive frontend and user interfaces.

Node.js

Provides the server-side JavaScript runtime environment.

Overall System Flow

React Frontend → Express.js API → Node.js Backend → Mongoose → MongoDB

For document-based question answering, the flow can be represented as:

User → React Interface → Q&A API → Document Retrieval → Relevant Passage Identification → Answer Generation → React Interface

When a user interacts with Pocket Lawyer, the React frontend sends requests to the Express.js backend.

Node.js processes the request and uses Mongoose to communicate with MongoDB.

For document questions, the backend retrieves the selected user's document, processes the relevant document content, and returns the answer through the API. The resulting data is returned through the API and rendered by the React interface.

This architecture provides a clear separation between the presentation layer, application logic, document processing, and data layer, making Pocket Lawyer easier to develop, maintain, test, and extend.

MODULES

Module 1: User Authentication & Management

This module handles user registration, login, and account management within the Pocket Lawyer platform. Users can create an account and log in to access their personalized legal document workspace.

Authentication provides the foundation for associating user activities with individual accounts. Once authenticated, users can upload documents, view their documents, ask questions, and access their personal history.

JWT-based authentication and password hashing can be used to secure user accounts and protected routes. The authentication system ensures that user-specific documents and private operations remain separated between different accounts.

Module 2: Document Upload & Management

The Document Upload and Management module allows users to upload and manage their legal documents.

Users can upload supported file formats including:

- PDF
- DOC
- DOCX
- TXT

The maximum supported file size is 10 MB. Uploaded documents are associated with the authenticated user and stored with relevant document information.

Users can view their uploaded documents and select a particular document for further interaction. This module forms the foundation of the document-based question-answering workflow.

Module 3: Document Text Extraction

The Document Text Extraction module processes uploaded files and extracts their readable text. After a document is uploaded, the backend processes the file and extracts its contents. The extracted text is stored along with the document record in MongoDB.

This extracted information is later used by the question-answering system to identify relevant sections of the document.

The module separates file processing from the frontend interface, allowing the React application to focus on user interaction while the backend handles document processing.

Module 4: Document-Based Question Answering

The Document-Based Question Answering module provides the primary intelligent feature of Pocket Lawyer. Users can select an uploaded document and ask questions related to its content.

The backend receives the document identifier and question and retrieves the corresponding document. Relevant passages are identified from the stored document text. The retrieved information is then used to generate a focused answer.

The result is returned to the React frontend and displayed to the user. Questions and answers can also be stored in MongoDB, allowing users to review their previous interactions with the document. This module connects document processing with an interactive question-answering workflow.

Module 5: Legal Topics & Search

The Legal Topics and Search module provides users with access to structured legal information. Users can browse or search available legal topics through the application.

This module complements document-based question answering by allowing users to explore legal concepts separately from their uploaded documents. Legal topic information is stored in MongoDB and can be retrieved through backend APIs. The React frontend displays the available topics in an organized interface.

Module 6: Bookmarks & Search History

The Bookmarks and Search History module helps users organize their interactions with the application.

Users can save useful legal topics or information through bookmarks. Search history maintains records of previous searches performed by the user. This information is associated with the user's account and stored in MongoDB.

The module allows users to revisit previously accessed information without repeating the same search.

Module 7: User Dashboard & Document Workspace

The User Dashboard provides a centralized view of the user's Pocket Lawyer activity.

The dashboard can display:

- Uploaded documents
- Recent questions
- Legal topics
- Bookmarked information
- Search history
- Document actions

The dashboard connects the major features of the application into a single workspace.

Users can upload a document, select an existing document, ask questions, and review their previous interactions through the dashboard. This module provides a central interface for managing the user's legal document activities.

Module Integration:

The seven modules work together as a connected system rather than as independent features.

The overall workflow can be represented as:

User Authentication → Upload Document → Extract Text → Select Document → Ask Question → Retrieve Relevant Content → Generate Answer → Save Q&A / History

The Authentication module establishes the user's identity.

The Document Management module handles uploaded documents.

The Text Extraction module prepares document content for processing.

The Q&A module uses the extracted document content to answer user questions.

The Legal Topics module provides additional structured legal information.

Bookmarks and Search History allow users to save and revisit useful information.

The Dashboard connects these features into a single user workspace.

All these modules are connected through the MERN architecture, with React handling user interaction, Node.js and Express.js providing backend APIs, and MongoDB with Mongoose handling persistent data storage.

FRONTEND CODE

Application Entry & Authentication

```
import React from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';
import { AuthProvider } from '@/context/AuthContext';
import './index.css';

createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <AuthProvider>
      <App />
    </AuthProvider>
  </React.StrictMode>
);
```

Application Routing & Navigation

```
export default function App() {
  return (
    <BrowserRouter>
      <Toaster position="bottom-right" />

      <AppShell>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/explore" element={<Explore />} />
          <Route path="/search" element={<SearchPage />} />
          <Route path="/topic/:id" element={<TopicDetail />} />
          <Route path="/dashboard" element={<Dashboard />} />
          <Route path="/documents" element={<Documents />} />
          <Route path="/documents/:id" element={<DocumentDetail />} />
          <Route path="/profile" element={<Profile />} />
          <Route path="/login" element={<Login />} />
          <Route path="/register" element={<Register />} />
        </Routes>
      </AppShell>
    </BrowserRouter>
```

```
  );
 }
```

User Authentication Management:

```
 const AuthContext = createContext(null);

 export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] =
   useState(Boolean(getToken()));

  const login = async (values) => {
   const data = await api.post("/auth/login", values);
   setToken(data.token);
   setUser(data.user);
   return data.user;
  };

  const register = async (values) => {
   const data = await api.post("/auth/register", values);
   setToken(data.token);
   setUser(data.user);
   return data.user;
  };

  const logout = () => {
   setToken(null);
   setUser(null);
  };

  return (
   <AuthContext.Provider
    value={{
     user,
     loading,
     isAuthenticated: Boolean(user),
     login,
     register,
     logout
    }}
   >
    {children}
   </AuthContext.Provider>
  );
}
```

Common Layout & Navigation:

```
export function AppShell({ children }) {
  const [location] = useLocation();
  const [mobileOpen, setMobileOpen] = useState(false);

  const { user, isAuthenticated, logout } = useAuth();

  const links = [
    ["/", "Home"],
    ["/explore", "Explore"],
    ["/search", "Search"],
    ["/about", "About"]
  ];

  return (
    <div className="app-shell">
      <header className="site-header">
        {/* Navigation and authentication */}
      </header>

      <main>{children}</main>

      <footer className="site-footer">
        {/* Footer links and information */}
      </footer>
    </div>
  );
  }
```

Legal Topic Display

```
export function TopicCard({ topic }) {
  return (
    <Link
      href={`/topic/${topic._id || topic.slug}`}
      className="topic-card"
    >
      <span>{topic.category}</span>
      <h3>{topic.title}</h3>
      <p>{topic.description}</p>

      <div className="tag-row">
        {(topic.tags || [])
          .slice(0, 3)
          .map((tag) => (
            <span className="tag" key={tag}>
```

```
{tag}
            </span>
          ))}
      </div>
    </Link>
  );
}
```

TopicCard

```
export function TopicCard({ topic }) {
  return (
    <Link href={`/topic/${topic._id || topic.slug}`}
      className="topic-card">
      <span>{topic.category}</span>
      <h3>{topic.title}</h3>
      <p>{topic.description}</p>
      <div className="tag-row">
        {(topic.tags || [])
          .slice(0, 3)
          .map((tag) => (
            <span className="tag" key={tag}>
              {tag}
            </span>
          ))}
      </div>
    </Link>
  );
}
```

BACKEND CODE

Server:

```
import 'dotenv/config';
import express from 'express';
import cors from 'cors';
import { createServer } from 'node:http';
import { connectDB } from './server/config/db.js';
import apiRouter from './server/routes/index.js';
import { errorMiddleware } from './server/middleware/errorMiddleware.js';
const app = express();
app.use(cors({ origin: process.env.CLIENT_ORIGIN || true }));
app.use(express.json({ limit: '2mb' }));
app.use('/uploads', express.static('server/uploads'));
app.use('/api', apiRouter);
app.use(errorMiddleware);
const port = Number(process.env.PORT || 3000);
await connectDB();
createServer(app).listen(port, '0.0.0.0',
  () => console.log(`Server running on port ${port}`)
);
```

Authentication:

```
import jwt from 'jsonwebtoken';
import User from '../models/User.js';
export async function authMiddleware(req, res, next) {
  try {
    const header = req.headers.authorization || '';
    const token = header.startsWith('Bearer ')
      ? header.slice(7)
      : req.cookies?.token;
    if (!token)
      return res.status(401).json({
        message: 'Authentication required.'
      });
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(payload.id);
```

```
    if (!user)
      return res.status(401).json({
        message: 'Session is no longer valid.'
      });
    req.user = user;
    next();
  } catch {
    res.status(401).json({
      message: 'Authentication required.'
    });
  }
}
```

Document Upload & Text Extraction

```
export async function uploadDocument(req, res) {
  if (!req.file || !allowed.has(req.file.mimetype))
    return res.status(400).json({
      message: 'Upload a PDF, DOC, DOCX, or TXT file.'
    });

  const doc = await Document.create({
    userId: req.user._id,
    fileName: req.file.originalname,
    fileType: req.file.mimetype,
    filePath: req.file.path,
    status: 'processing'
  });

  try {
    const extractedText = await extractText(
      req.file.path,
      req.file.mimetype,
      req.file.originalname
    );

    doc.extractedText = extractedText.slice(0, 2000000);
    doc.status = 'ready';

    await doc.save();
    res.status(201).json(doc);
  } catch {
    doc.status = 'failed';
    await doc.save();
```

```
 res.status(422).json({
      message: 'Document processing failed.'

    });
  }
}
```

Document-Based Q&A:

```
export async function askQuestion(req, res) {
  const question = String(req.body.question || '').trim();

  if (question.length < 3)
    return res.status(400).json({
      message: 'Ask a more specific question.'
    });

  const doc = await Document.findOne({
    _id: req.params.id,
    userId: req.user._id
  });

  if (!doc)
    return res.status(404).json({
      message: 'Document not found.'
    });

  if (doc.status !== 'ready')
    return res.status(409).json({
      message: 'This document is not ready for questions.'
    });

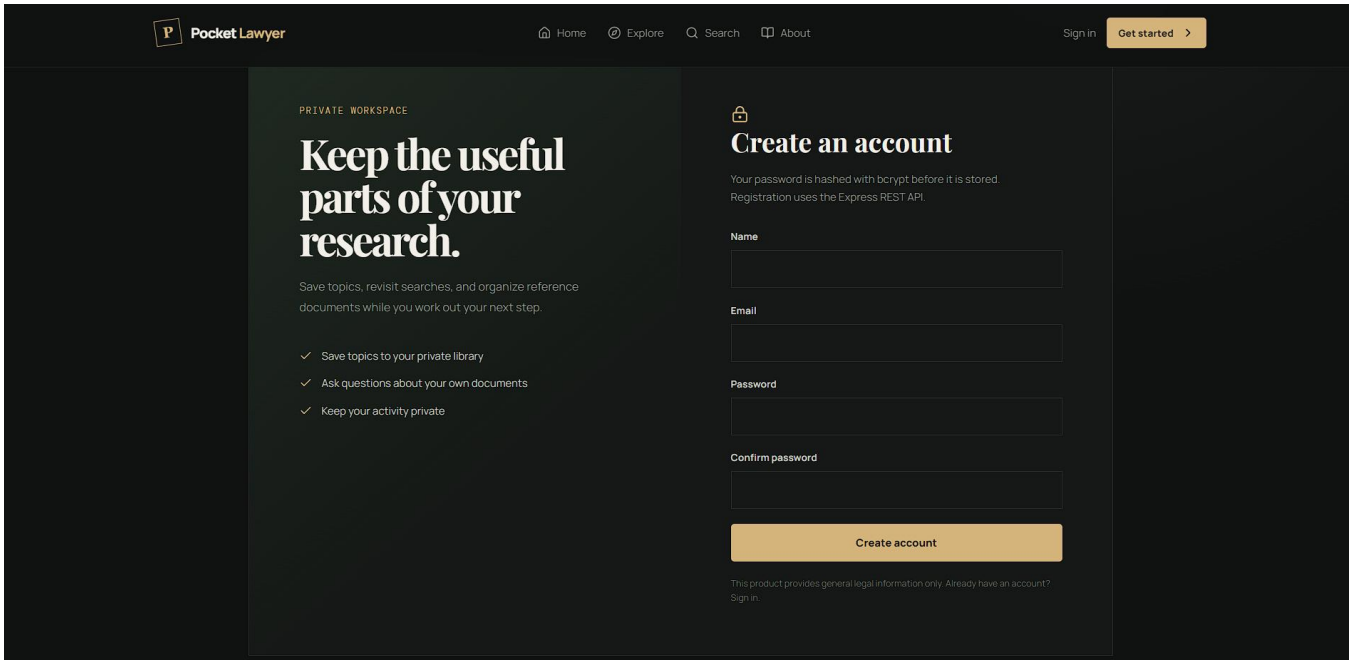
  const result =
    answerFromDocument(question, doc.extractedText);

  const saved = await DocumentQuestion.create({
    documentId: doc._id,
    userId: req.user._id,
    question,
    ...result
  });

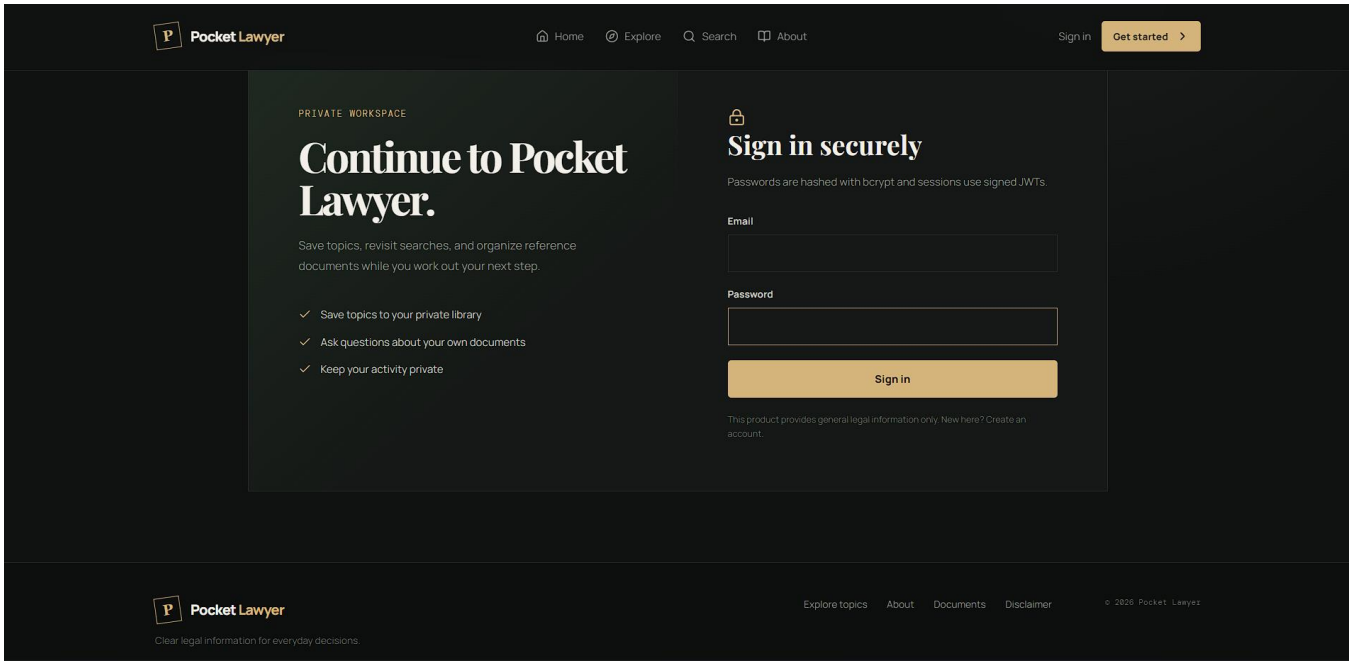
  res.status(201).json(saved);
}
```

OUTPUT

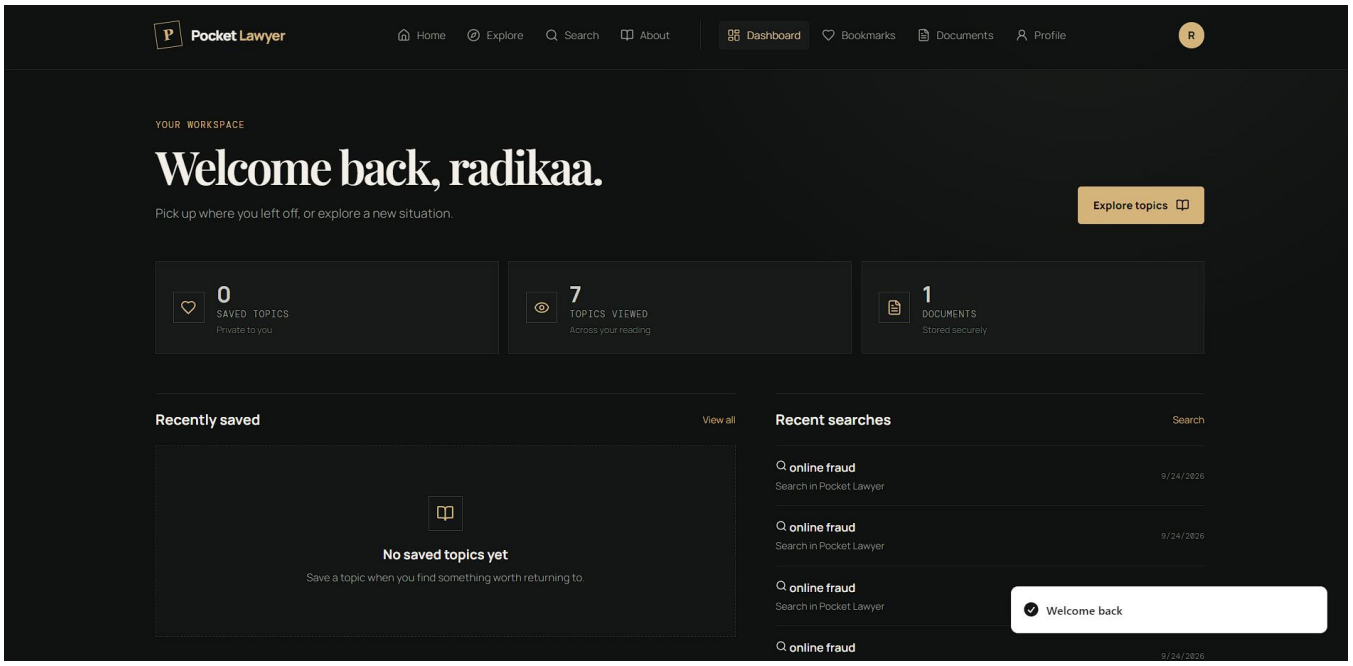
Register Page:



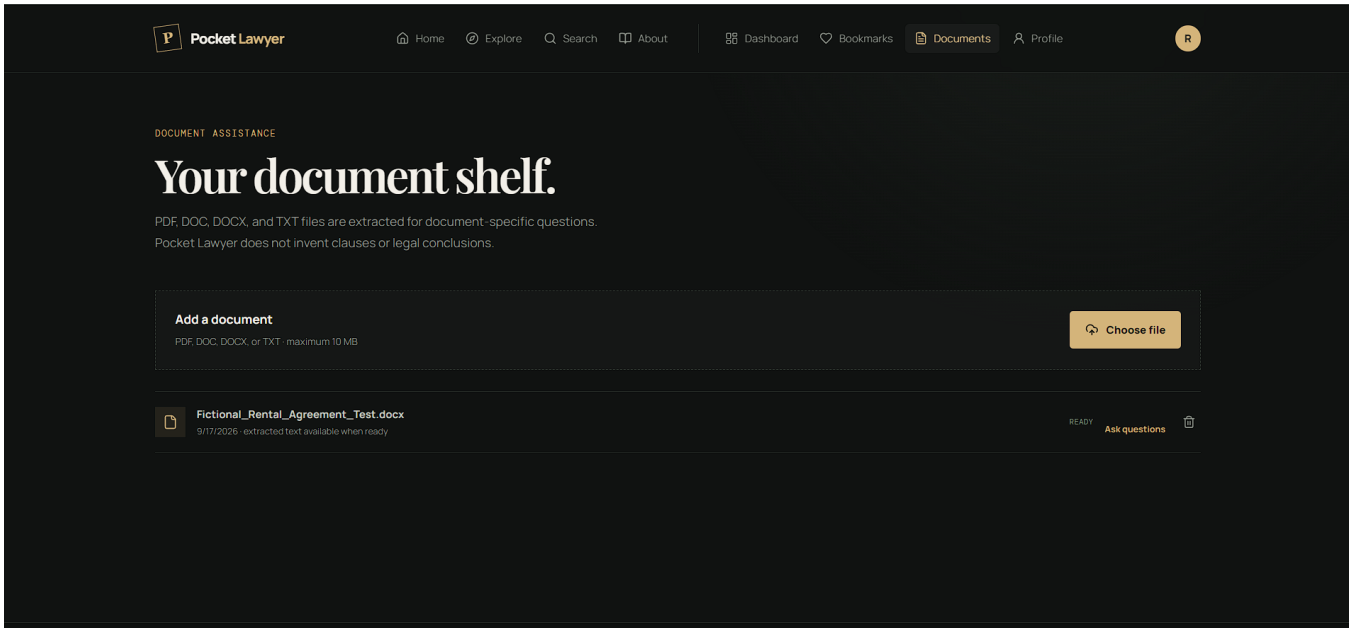
Login Page:



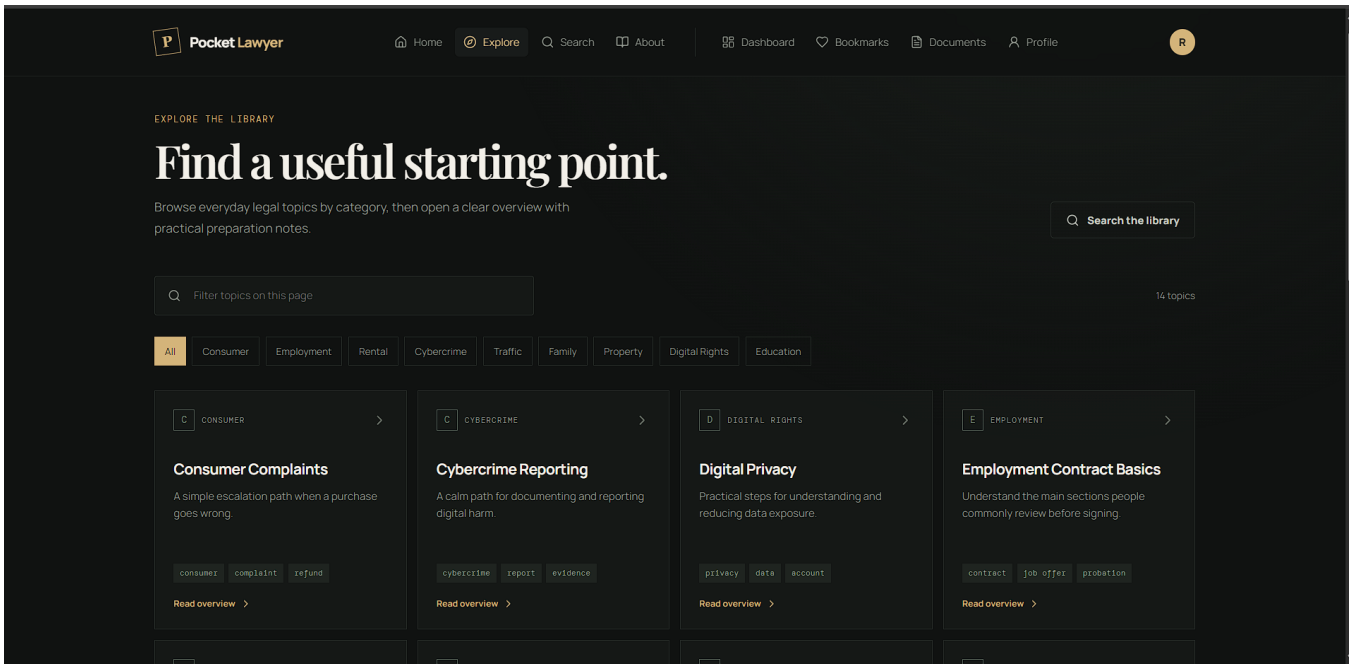
Dashboard:



Document-Based Q&A:



Topics:


Pocket Lawyer
Home
Explore
Search
About
Dashboard
Bookmarks
Documents
Profile
R
EXPLORE THE LIBRARY
Find a useful starting point.
Browse everyday legal topics by category, then open a clear overview with practical preparation notes.
Search the library
Filter topics on this page
14 topics
All
Consumer
Employment
Rental
Cybercrime
Traffic
Family
Property
Digital Rights
Education
C CONSUMER
Consumer Complaints
A simple escalation path when a purchase goes wrong.
consumer complaint refund
Read overview
C CYBERCRIME
Cybercrime Reporting
A calm path for documenting and reporting digital harm.
cybercrime report evidence
Read overview
D DIGITAL RIGHTS
Digital Privacy
Practical steps for understanding and reducing data exposure.
privacy data account
Read overview
E EMPLOYMENT
Employment Contract Basics
Understand the main sections people commonly review before signing.
contract job offer probation
Read overview